
Modül 08 — Redis ve Önbellekleme

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Ürün Detay Önbelleği (Kolay)	2
Ödev 2 — Oyun Liderlik Tablosu (Orta)	3
Ödev 3 — İstek Sınırlayıcı (Rate Limiter) (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/08-redis-caching/exercise/` altındadır. TODO yorumlarını bulun.
2. Testler gerçek bir Redis'e (Testcontainers) karşı çalışır. **Docker açık olmalıdır.**
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/08-redis-caching/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/08-redis-caching/solution/` altındaki çözümü okuyun.

★ İpucu

Ödevler birbirinden bağımsızdır. Her test sınıfı boş bir önbellekle başlar veya kendi anahtarlarını siler.

Ödev 1 — Ürün Detay Önbelleği (Kolay)

Hedef: Ürün detaylarını Redis'ten sunmak ve bir güncellemeden sonra asla eski fiyatı göstermemek.

ProductCatalog hazır verilmiştir. Yavaştır (okuma başına 200 ms) ve okumalarını sayar. Böylece testler önbelleğin kullanılıp kullanılmadığını görebilir.

Yapılacaklar (exercisel paketi):

- TODO 1a — `ProductDetailService.detail`: Sonucu `product-details` önbelleğine alın.
- TODO 1b — `ProductDetailService.changePrice`: Değişiklikten sonra önbellekten **yalnızca bu ürünün** kaydını silin.
- TODO 1c — `CacheConfiguration`: `product-details` önbelleğinin TTL'i 5 dakika olsun, `ProductDetail` tipinde JSON saklasın ve null değer saklamasın.

İpuçları:

- Ders bölüm 3.1: `@Cacheable(cacheNames = "...")`.
- Ders bölüm 3.3: `@CacheEvict(cacheNames = "...", key = "#id")`. key olmadan anahtar **iki** parametreden birden oluşur ve eşleşmez.
- Ders bölüm 3.2: Verilen builder'a `.withCacheConfiguration("product-details", RedisCacheConfiguration.defaultCacheConfig())` ekleyin.

Kabul kriterleri: `ExerciselTest` içindeki 4 testin hepsi geçer.

Tahmini süre: 25 dakika

Ödev 2 — Oyun Liderlik Tablosu (Orta)

Hedef: Oyuncuların canlı sıralamasını bir Redis sorted set’inde tutmak.

Yapılacaklar (exercise2.Leaderboard):

- TODO 2a — addPoints: Puanları oyuncunun skoruna ekleyin. Yeni bir oyuncu 0’dan başlar.
- TODO 2b — top: En iyi count oyuncuyu, en yüksek skor önce olacak şekilde PlayerScore kayıtları olarak döndürün.
- TODO 2c — rankOf: Oyuncunun sırası: lider için 1, ikinci için 2 ve böyle devam eder. Puanı olmayan oyuncu için boş.

İpuçları:

- Ders bölüm 3.4: opsForZSet().incrementScore(...) ve reverseRangeWithScores(key, 0, count - 1).
- opsForZSet().reverseRank(key, player) **0’dan başlar** ve bilinmeyen üye için null döndürür.
- Optional.ofNullable(...).map(...).

Kabul kriterleri: Exercise2Test içindeki 3 testin hepsi geçer.

Tahmini süre: 30 dakika

Ödev 3 — İstek Sınırlayıcı (Rate Limiter) (Zor)

Hedef: Her istemciye bir zaman penceresinde en fazla limit çağrı izni vermek. Örneğin bir API anahtarı için dakikada 100 çağrı.

Bu bir **sabit pencere** (fixed window) sınırlayıcıdır. Zaman eşit uzunlukta pencerelere bölünür ve her istemcinin pencere başına bir sayacı vardır. Yeni bir pencere başladığında yeni bir sayaç 0’dan başlar.

Yapılacaklar (exercise3.RateLimiter.tryAcquire):

- TODO 3a — İstemci ve pencere başına bir anahtar: rate:<clientId>:<pencere numarası>. Pencere numarası clock.millis() / window.toMillis() olur.
- TODO 3b — Çağrıyı Redis’te atomik olarak sayın.
- TODO 3c — Bir penceredeki ilk çağrı, anahtarın bir pencere süresi sonra silinmesini sağlasın. Böylece Redis eski sayaçları kaldırır.
- TODO 3d — Sayı limit içinde olduğu sürece true döndürün.

İpuçları:

- Ders bölüm 3.5: opsForValue().increment(key) yeni değeri döndürür. Yalnızca 1 alan çağrının süre ayarlaması gerekir.

- Zamanı her zaman enjekte edilen `Clock`'tan alın, asla `System.currentTimeMillis()`'ten değil. Test saati ileri sarar.
- `redis.expire(key, window)`.

Kabul kriterleri: Exercise3Test içindeki 4 testin hepsi geçer.

Tahmini süre: 40 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Sabit pencere, pencere sınırı civarında limitin iki katına kadar çağrıya izin verir (bir pencerenin sonu artı sonrakinin başı). Bunun yerine bir **kayan pencere** (sliding window) sınırlayıcı yazın: Her çağrının zaman damgasını bir sorted set'te saklayın, bir pencereden eski kayıtları `ZREMRANGEBYSCORE` ile silin ve kalanları `ZCARD` ile sayın. Bu adımlar neden bir Lua script'inde veya bir `MULTI/EXEC` transaction'ında çalışmalıdır?